AgentOS

The Operating System for Autonomous AI Agents

Architecture Design Document · Software Design Specification

Document Classification: Internal Engineering — Architecture Review

Prepared for: Senior Engineering Leadership & Platform Architecture Review Board

Version 1.0

Table of Contents

1. Executive Summary

1.1 Vision

AgentOS is the runtime layer that autonomous AI agents execute on top of — the same way Kubernetes is the runtime layer that containers execute on top of, and Linux is the runtime layer that processes execute on top of. Where a container packages code and a process packages a running program, an agent packages a goal, a set of tools, a memory, and a policy for making decisions over time. Today there is no shared substrate for running that unit reliably at scale. AgentOS is designed to be that substrate.

1.2 Mission

Give every engineering organization the ability to deploy, schedule, observe, secure, and scale AI agents with the same operational maturity that Kubernetes brought to microservices — without every team having to reinvent scheduling, memory, tool sandboxing, retries, and cost governance from scratch.

1.3 Why AgentOS Exists

Every team building agentic systems today converges on the same list of unsolved infrastructure problems: agents that run for hours or days need durable state and checkpointing; agents that call external tools need sandboxing and permissioning; agents that call LLMs need routing, fallback, and cost control; agents that talk to other agents need a communication substrate; and all of this needs to be observable, or the moment an agent misbehaves in production nobody can explain why. Right now, every company solves these problems with bespoke glue code sitting on top of a workflow orchestrator (Airflow, Temporal), a vector database, and a hand-rolled scheduler. That glue code is not a product a team wants to own — it is undifferentiated infrastructure, and undifferentiated infrastructure is exactly what should be standardized into a platform.

1.4 Why Current Frameworks Are Insufficient

Framework	What it actually is	Where it breaks down at scale
LangChain / LangGraph	A composition library and a graph-execution library for building single-process agent logic	No native multi-tenant scheduling, no cluster-wide resource management, no first-class permission model, checkpointing is bolted on, not designed for thousands of concurrent long-running agents
CrewAI	A role-based multi-agent orchestration library	Optimized for scripted, small-team agent crews; lacks a durable runtime, a real scheduler, or production-grade isolation between agents and tools
AutoGen	A conversation-pattern framework for multi-agent dialogues	Conversation-centric, not workflow- or state-centric; weak story for long-running, checkpointable, resumable

Framework	What it actually is	Where it breaks down at scale
		execution across process or node restarts
Semantic Kernel	An SDK for orchestrating skills/plugins around an LLM	SDK-shaped, not platform-shaped: no cluster scheduler, no built-in multi-tenant memory or cost governance
Temporal / Airflow	General-purpose durable workflow engines	Excellent at DAGs and durability, but have no concept of an 'agent' as a first-class entity: no agent memory, no LLM gateway, no tool sandbox, no agent-to-agent messaging

The common thread: these are libraries and orchestrators an application developer imports into their own process. None of them is a multi-tenant, cluster-aware, security-isolated runtime that treats the agent itself — its memory, its identity, its permissions, its cost budget — as the unit of deployment. AgentOS is designed to be that runtime, with these frameworks welcome to run as workflow definitions or SDKs on top of it.

1.5 Target Users

- **Platform teams —** inside large enterprises who need to give hundreds of internal teams a safe, governed way to ship agents
- **AI-native startups —** building agent products (coding agents, research agents, support agents) who don't want to build their own scheduler and memory system
- **Regulated industries —** (finance, healthcare, government) that require audit trails, RBAC/ABAC, and sandboxed tool execution before any agent can touch production systems
- **System integrators / consultancies —** deploying agent solutions across many clients and needing a repeatable operational model

1.6 Target Industries

- Software engineering & DevOps automation
- Financial services (research, compliance, fraud)
- Healthcare (diagnostics support, clinical documentation)
- Customer support & operations
- Cybersecurity operations centers
- Scientific research & pharma R&D
- Enterprise knowledge management

WHY THIS EXISTS

Frameworks answer 'how do I write agent logic.' AgentOS answers 'how do I run thousands of agent instances, from many teams, safely, cheaply, and observably, forever.' Those are different problems, and the second one has no incumbent.

2. Core Philosophy

2.1 Every Compute Era Gets an Operating System

Infrastructure history is a sequence of 'unit of work' abstractions each getting their own runtime once they became common enough to standardize:

Compute unit	Runtime that emerged	What the runtime standardized
Process	Linux / Unix kernel	Scheduling, memory isolation, syscalls, file descriptors
Container	Docker	Packaging, image layers, namespaces/cgroups isolation
Cluster of containers	Kubernetes	Scheduling, self-healing, service discovery, rolling upgrades
Batch job / DAG	Apache Airflow	Dependency graphs, retries, scheduling of data pipelines
Durable function	Temporal	Checkpointed, resumable long-running business logic
Microservice mesh	Istio	Traffic policy, mTLS, observability between services
Autonomous agent	AgentOS (this document)	Agent lifecycle, memory, tool permissions, LLM routing, agent-to-agent messaging, cost governance

An agent is not a container and not a DAG run. It is a long-lived, stateful, non-deterministic decision-maker that consumes a probabilistic model as its 'CPU,' reads and writes memory across sessions, and takes actions in the world through tools with real consequences. That combination — statefulness, non-determinism, and the ability to cause side effects — is precisely why agents need their own runtime rather than being wedged into a container scheduler or a data-pipeline DAG engine.

2.2 Design Tenets

- **Agents are first-class resources —** not application code. AgentOS schedules, versions, and secures an agent the way Kubernetes schedules, versions, and secures a Pod.
- **Non-determinism is expected, not an edge case —** Every subsystem (scheduler, workflow engine, memory) is built assuming an LLM call can return something unexpected, and failure/compensation paths are first-class, not an afterthought.
- **Memory is infrastructure, not application state —** Just as Kubernetes doesn't make every app implement its own DNS, AgentOS provides memory as a managed, multi-tiered service.
- **Tools are the blast radius —** the security model is built around the assumption that most damage an agent can do happens through a tool call, so tool execution gets the most isolation and auditing.

- **Everything is observable by default —** every LLM call, tool call, memory read/write and inter-agent message is traced, because debugging a probabilistic system without full replay is nearly impossible.
- **Cost is a scheduling dimension —** token spend and compute spend are treated the way CPU/memory requests are treated in Kubernetes — declared, budgeted, and enforced.

WHY THIS EXISTS

If any one of these tenets is missing, teams end up rebuilding it themselves per-project, which is exactly the fragmentation AgentOS exists to end.

3. Functional Requirements

AgentOS must provide the following capabilities as first-class, API-driven platform features rather than patterns developers re-implement.

3.1 Agent Lifecycle & Scheduling

- Dynamic agent creation from a declarative Agent Manifest (analogous to a Pod spec)
- Full lifecycle management: register, initialize, run, sleep, wake, migrate, upgrade, roll back, terminate, garbage collect
- Cluster-aware scheduling of agent instances across worker nodes with resource, affinity, and priority constraints
- Horizontal auto-scaling of agent pools based on queue depth, latency, and cost signals

3.2 Memory & State

- Multi-tier memory: working, episodic, semantic, and shared/global memory
- Durable checkpointing of agent state for pause/resume and crash recovery
- Context compression and summarization services for long-running sessions

3.3 Communication

- Direct agent-to-agent request/response messaging
- Publish/subscribe and broadcast channels for event-driven coordination
- Durable event streaming for audit and replay

3.4 Workflow Orchestration

- DAG- and state-machine-based multi-step workflows spanning multiple agents and tools
- Retries, compensation (saga-style rollback), conditional branching, loops, and timeouts
- Native human-in-the-loop approval steps

3.5 Tool Execution

- Sandboxed execution environment per tool invocation
- A permission model that scopes exactly which tools/resources an agent identity may use
- Full audit logging of every tool call, input, and output

3.6 Platform Services

- Authentication and authorization (RBAC/ABAC) for humans, agents, and services
- Observability: logging, metrics, distributed tracing, cost dashboards, execution replay
- Agent, tool, and workflow versioning with safe rollout and rollback
- Rate limiting, cost limits, and resource quotas per tenant/team/agent
- A plugin/marketplace system for tools, memory modules, and workflow templates
- Distributed, multi-node execution with failover

> **NOTE**

> Sections 6 onward map each of these requirements to a concrete component and explain the reasoning behind its design.

4. Non-Functional Requirements

Dimension	Target	Design implication
Latency	P50 scheduling overhead < 50ms; tool call dispatch < 100ms excluding tool runtime	Scheduler and gateway kept on the hot path lean; heavy work (memory retrieval, embeddings) is async or cached
Scalability	100,000+ concurrent agent instances per cluster; horizontal scale-out of every stateless tier	Stateless control-plane services behind the API Gateway; sharded task queues; partitioned memory store
Fault tolerance	No single node failure loses agent progress beyond the last checkpoint interval	Mandatory checkpointing, leader election for schedulers, replicated event log
Availability	99.9%+ for control plane, degrade-gracefully for data plane under partial outage	Control plane and data plane deployed and failure-isolated separately
Consistency	Strong consistency for agent state transitions; eventual consistency acceptable for memory search indexes	Agent state in a consensus-backed store (e.g., etcd/Raft-based); vector indexes rebuilt asynchronously
Durability	Zero data loss for committed checkpoints and audit events	Write-ahead event log, replicated storage, checkpoint-before-acknowledge semantics
Security	Least-privilege by default, full audit trail, sandboxed tool execution	Per-agent identity, per-tool permission scopes, sandboxed runtimes (gVisor/Firecracker-class isolation)
Extensibility	New tools, memory backends, and LLM providers addable without core redeploy	Plugin registry with versioned, sandboxed plugin execution and a stable extension API
Portability	Runnable on any Kubernetes-conformant cluster, on-prem or cloud	Core control plane packaged as Kubernetes operators/CRDs; no hard cloud-vendor lock-in

TRADE-OFF

Strong consistency everywhere would be simpler to reason about but would throttle throughput on the hottest paths (memory writes, event ingestion). AgentOS instead uses strong consistency only where correctness of agent state transitions is at stake, and eventual consistency for search/indexing paths where staleness of a few seconds is acceptable.

5. High-Level Architecture

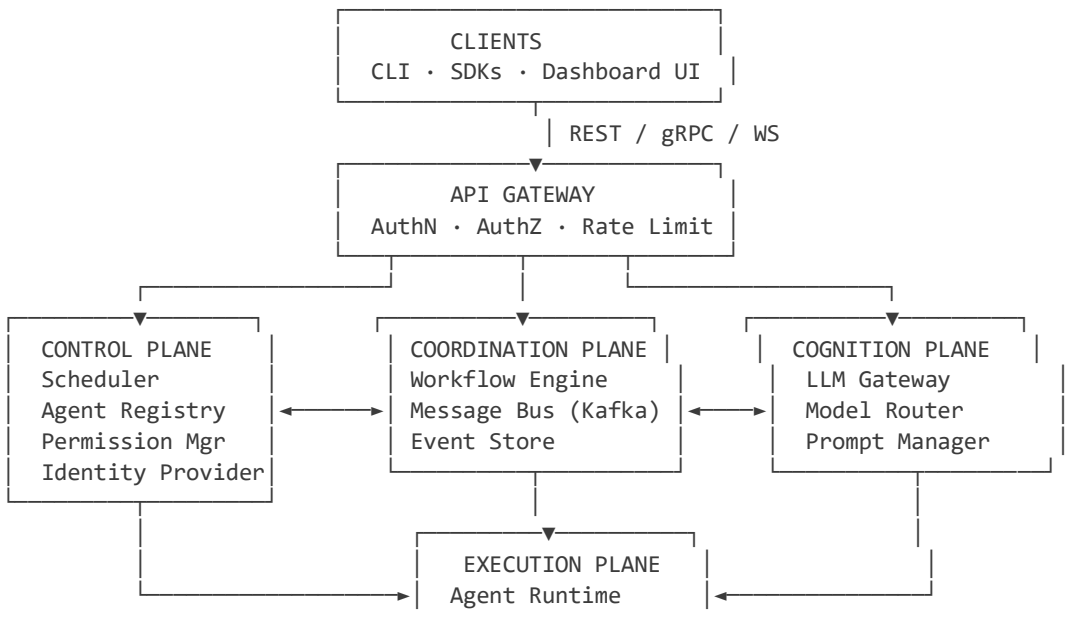
AgentOS is organized into four planes, mirroring the control-plane/data-plane split that made Kubernetes operable at scale, plus two planes unique to agentic workloads: a Cognition Plane (LLM access) and a Memory Plane.

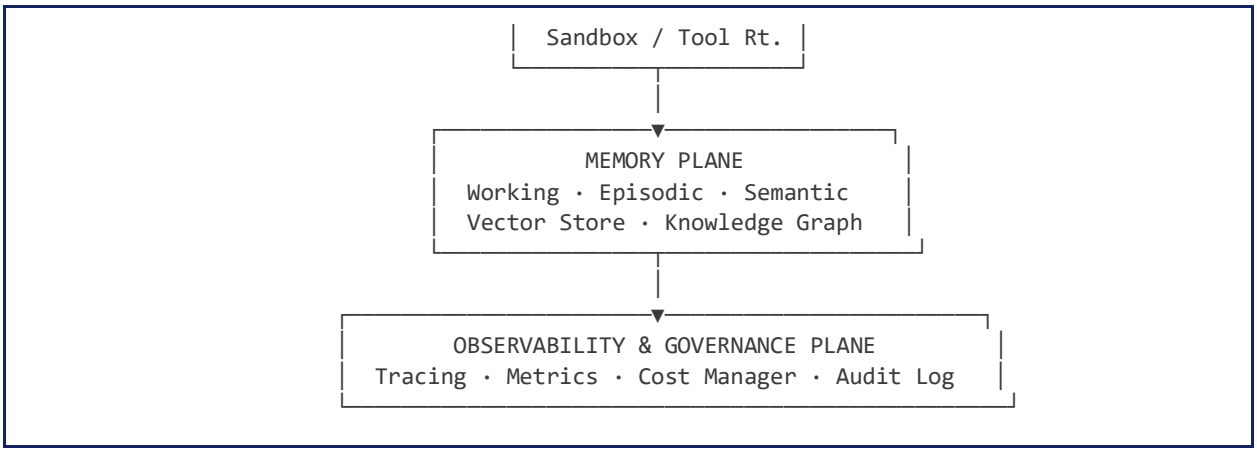
5.1 The Six Planes

Plane	Responsibility	Analogy
Control Plane	API Gateway, Scheduler, Agent Registry, Policy/Permission Manager, Identity Provider	Kubernetes control plane (API server, scheduler, controller manager)
Execution Plane	Agent Runtime workers, Sandboxes, Tool Runtime	Kubernetes kubelet + container runtime
Cognition Plane	LLM Gateway, Model Router, Prompt Manager	GPU/accelerator abstraction layer
Memory Plane	Working/episodic/semantic memory stores, Vector Store, Knowledge Graph	Distributed database layer
Coordination Plane	Message Bus, Event Store, Workflow Engine	Kafka + Temporal combined
Observability & Governance Plane	Tracing, Metrics, Cost Manager, Dashboards, Audit Log	Prometheus/Grafana + a compliance layer

5.2 System Diagram

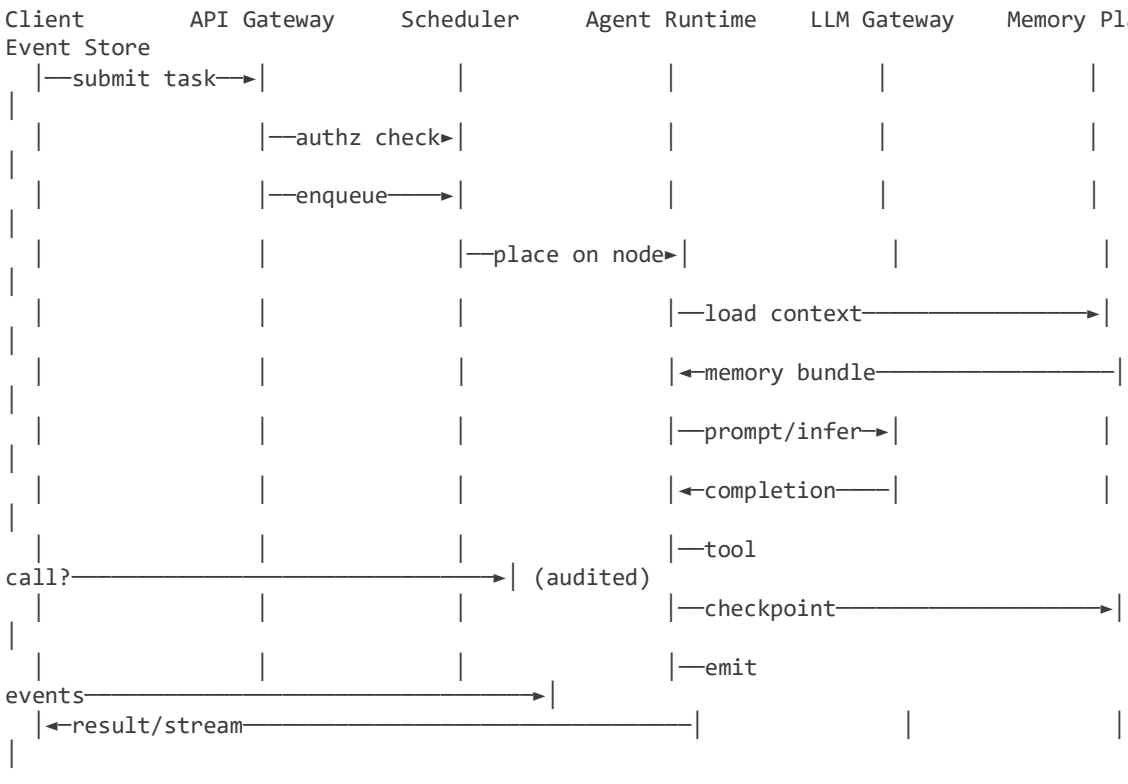
Figure 5.1 — AgentOS Top-Level Component Diagram





5.3 Request Flow — Sequence Overview

Figure 5.2 — Sequence: Client Submits a Task to an Agent



WHY THIS EXISTS

Separating planes lets each scale on its own axis: the Cognition Plane scales with token throughput, the Memory Plane scales with storage/QPS, and the Execution Plane scales with concurrent agent count. In a monolithic design these get coupled and you over-provision the cheapest resource to satisfy the most expensive one.

TRADE-OFF

Six planes means six sets of APIs and failure domains to operate. A smaller team might collapse Coordination and Cognition into the Execution Plane for a v1 to reduce operational surface area, accepting tighter coupling in exchange for a simpler initial rollout.

6. Component Deep Dive

6.1 API Gateway

Single entry point for all external traffic (REST, gRPC, WebSocket). Terminates TLS, authenticates callers via the Identity Provider, applies rate limits and quotas, and routes to the correct internal service. Stateless and horizontally scaled behind a load balancer.

> **TRADE-OFF**
> A single gateway is an extra network hop versus clients calling services directly, but it centralizes auth, quota, and audit logging in one place instead of duplicating that logic across every internal service.

6.2 Scheduler

Assigns queued agent tasks to worker nodes in the Execution Plane. Implements priority queues, fair-share scheduling across tenants, preemption of low-priority work, and affinity/anti-affinity rules (e.g., keep an agent co-located with its memory shard; keep two agents with conflicting tool locks apart). Modeled closely on the Kubernetes scheduler's predicate/priority-function design, adapted for token-budget and GPU/accelerator awareness.

6.3 Workflow Engine

Executes multi-step, multi-agent workflows expressed as DAGs or state machines, with durable checkpointing after every step so a crashed worker can resume exactly where it left off. Provides retries with backoff, saga-style compensation for partial failures, conditional branches, loops, timeouts, and native human-approval gates.

6.4 Execution Engine / Agent Runtime

The process (or Firecracker/gVisor micro-VM) that actually runs an agent's reasoning loop: pull context from memory, call the LLM Gateway, decide on an action, call the Tool Runtime if needed, write results back to memory, and emit events. Each agent instance runs isolated from every other agent instance, with resource limits enforced by cgroups-equivalent controls.

6.5 Memory Engine

A unified API in front of four memory tiers (working, episodic, semantic, shared) described fully in Section 9. Handles retrieval, ranking, consolidation, and expiration so individual agents don't need to reimplement retrieval-augmented generation plumbing.

6.6 Permission Manager & Identity Provider

Every human, service, and agent has a cryptographically verifiable identity (mutual TLS or signed JWT). The Permission Manager evaluates RBAC and ABAC policies before any tool call, memory access, or inter-agent message is allowed to execute, and logs the decision to the audit trail.

6.7 Message Bus & Event Store

A Kafka-class durable log is the backbone for both event streaming (audit, replay) and pub/sub coordination between agents. The Event Store is the system of record; the Message Bus is the delivery mechanism. See Section 8 for the full comparison of messaging technologies.

6.8 Tool Runtime

Executes tool calls (browser, database, filesystem, cloud APIs, Git, Slack, terminal) inside a locked-down sandbox with a scoped credential injected just-in-time, never given to the agent's own process. Every call is logged with input, output, and the permission decision that allowed it.

6.9 Prompt Manager

Central registry of versioned prompt templates, few-shot examples, and system instructions, decoupled from agent code so prompts can be updated, A/B tested, and rolled back without redeploying an agent.

6.10 LLM Gateway & Model Router

A single abstraction over every supported model provider. The Model Router selects a model per-call based on task complexity, latency budget, and cost budget, and can fall back to a secondary provider automatically on error or timeout. Detailed in Section 12.

6.11 Checkpoint Manager

Periodically (and after every workflow step) serializes agent state — memory pointers, workflow position, pending tool calls — to durable storage, enabling pause/resume, migration between nodes, and crash recovery without replaying an entire session from scratch.

6.12 Task Queue

Sharded, priority-aware queue sitting between the Scheduler and the Execution Plane, backed by the Message Bus, so tasks survive scheduler restarts.

6.13 Vector Store / Semantic & Episodic Memory Stores

Purpose-built stores for embedding search (semantic memory) and time-ordered interaction logs (episodic memory), both described in Section 9.

6.14 Agent Registry & Plugin Registry

The Agent Registry is the source of truth for every agent definition, version, and deployed instance — analogous to a container image registry plus a Kubernetes API server's object store. The Plugin Registry holds tools, memory modules, and workflow templates contributed by the marketplace ecosystem (Section 22).

6.15 Dashboard, Tracing, Monitoring, Cost Manager

The operator-facing surface: live agent execution views, distributed traces per task, Prometheus-style metrics, and a cost dashboard that attributes token and compute spend to team, agent, and workflow (Section 21).

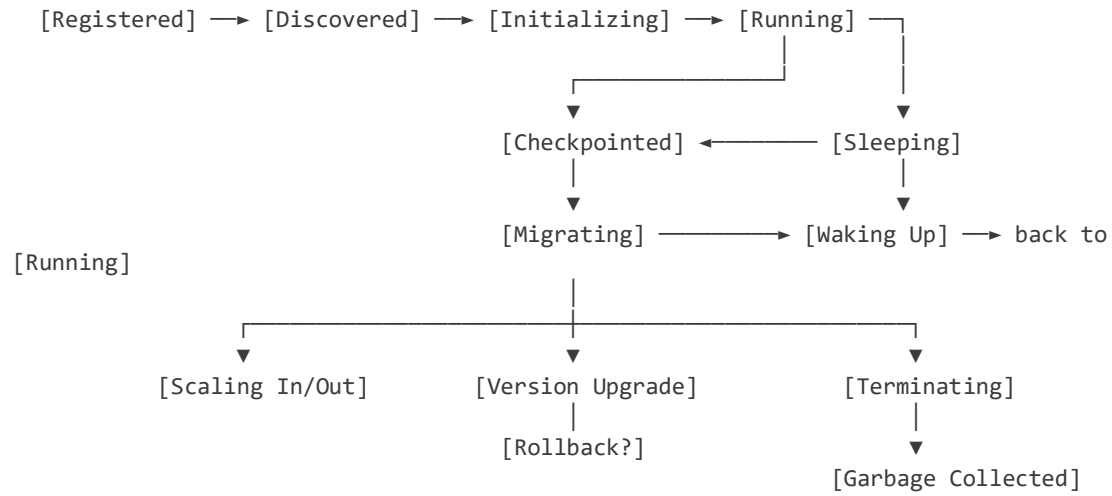
6.16 Storage Layer

Object storage for large artifacts and checkpoints, a relational store for registry/metadata, and a document/columnar store for events — chosen per the durability and query patterns in Section 16.

7. Agent Lifecycle

Every agent instance moves through a well-defined state machine, giving operators the same predictability Kubernetes gives Pods.

Figure 7.1 — Agent Lifecycle State Machine



7.1 Registration & Discovery

An agent is registered by submitting an Agent Manifest (YAML/JSON) declaring its model preferences, tool permissions, memory requirements, and resource limits to the Agent Registry. Discovery is the process by which the Scheduler and other agents locate available agent types and running instances via the registry's service-discovery API.

7.2 Initialization

On placement, the runtime pulls the agent's prompt bundle, tool permission set, and any existing memory checkpoint, then boots the sandboxed execution environment. Initialization is idempotent so repeated retries after a failed start don't corrupt state.

7.3 Execution & Checkpointing

While running, the agent processes tasks from its queue. After each meaningful step (a tool call, a workflow transition, a memory write) the Checkpoint Manager persists enough state to resume without replay. This turns a multi-hour agent session into a sequence of small, recoverable steps rather than one long, fragile process.

7.4 Sleeping & Wake-up

Agents waiting on a long external event (human approval, a webhook, a scheduled follow-up) release their compute slot and persist to a 'sleeping' state, waking on the triggering event. This is essential for cost efficiency — a support agent waiting three days for a customer reply should consume zero compute in the meantime.

7.5 Migration & Scaling

Because state lives in checkpoints rather than only in worker memory, an agent instance can be migrated to a different node (for rebalancing or node maintenance) or an agent pool can scale horizontally by spinning up new instances from the same manifest and letting the Scheduler distribute the queue across them.

7.6 Version Upgrades & Rollback

Agent manifests are versioned. AgentOS supports canary rollout (a percentage of new tasks routed to v2 while v1 keeps serving), and automatic rollback if v2's error rate or cost exceeds a declared threshold — the same pattern as a Kubernetes Deployment rollout, applied to agent versions instead of container images.

7.7 Termination, Recovery & Garbage Collection

Graceful termination flushes final checkpoints and emits a terminal event. Crash recovery restarts from the last checkpoint on a healthy node. Garbage collection reclaims sandboxes, ephemeral storage, and expired checkpoints per the tenant's retention policy.

> **WHY THIS EXISTS**
>
> Treating lifecycle as a formal state machine — rather than 'a Python process that runs until it doesn't' — is what makes agents restartable, migratable, and auditable. It's the single biggest reason today's frameworks struggle in production: they model an agent as a function call, not a managed resource.

8. Agent Communication

Agents coordinate through five patterns, each suited to a different interaction shape:

Pattern	Use case	Delivery guarantee
Request/Response	Agent A needs a synchronous answer from Agent B (e.g., 'summarize this document')	At-most-once, with client-side retry
Direct messaging	Targeted async handoff between two known agents	At-least-once, durable
Publish/Subscribe	One agent's output is relevant to many subscribers (e.g., 'new ticket created')	At-least-once, fan-out
Broadcast	Control-plane signals to all agents of a type (e.g., 'drain before upgrade')	Best-effort, idempotent handlers required
Shared memory	Multiple agents collaborating on the same working context (e.g., a research crew)	Read-your-writes within a session

8.1 Message Bus Technology Choice

Option	Strengths	Weaknesses for AgentOS
Kafka	Durable log, strong ordering per partition, huge ecosystem, replay for audit/debug	Higher operational complexity, higher latency floor than NATS
RabbitMQ	Mature, flexible routing (exchanges), good for complex routing topologies	Weaker as a durable audit log; scaling to very high throughput is harder
NATS / NATS JetStream	Extremely low latency, simple operations, good pub/sub semantics	Smaller ecosystem for long-term retention and replay tooling than Kafka
Redis Streams	Very low latency, simple to run, good for ephemeral coordination	Not designed as a durable system-of-record for years of audit history

AgentOS uses Kafka (or a Kafka-API-compatible system such as Redpanda) as the Event Store / durable backbone, and layers a lighter-weight NATS deployment for latency-sensitive direct messaging and broadcast between co-located agents. This mirrors how many large systems separate 'the log of record' from 'the low-latency control channel' rather than forcing one technology to do both jobs well.

> **TRADE-OFF**
> Running two messaging systems is more operational surface than picking one. The alternative — Kafka only — is viable for a v1 platform and removes an entire technology from the stack at the cost of slightly higher p50 latency for tight agent-to-agent loops.

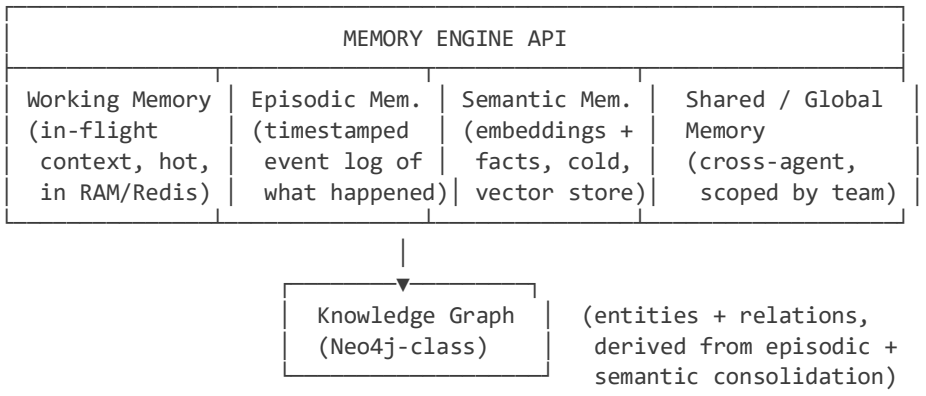
8.2 Shared Memory Coordination

For tightly-coupled agent crews (e.g., a planner and several worker agents), AgentOS exposes a scoped 'session memory' region in the Memory Plane that multiple agent instances can read and write with optimistic concurrency control, avoiding the overhead of routing every intermediate thought through the message bus.

9. Memory Architecture

Memory is the single most differentiating subsystem in AgentOS versus a generic workflow engine. It is modeled as four tiers plus a supporting knowledge graph, exposed through one Memory Engine API so agent authors never talk to raw storage directly.

Figure 9.1 — Memory Tiers



9.1 Working Memory

The agent's current context window contents: recent turns, tool results, scratchpad reasoning. Backed by an in-memory store (Redis-class) for low latency, with a TTL and a hard size cap that triggers compression before the context is sent to the LLM.

9.2 Episodic Memory

An append-only, timestamped log of everything the agent has done: tasks received, decisions made, tool calls, outcomes. This is the raw material for audit, debugging replay, and for later consolidation into semantic memory. Stored in a columnar/document store optimized for time-range queries.

9.3 Semantic Memory

Durable facts and preferences extracted from episodic memory, stored as embeddings in a vector store for similarity search (e.g., 'what do we know about this customer') and, where structure is known, as rows/entities for exact lookup. This is what most people mean by 'RAG memory.'

9.4 Shared / Global Memory

A namespace visible to a defined group of agents (a team, a workflow) for collaboration — the mechanism referenced in Section 8.2's shared-memory coordination pattern.

9.5 Knowledge Graph

Entities (people, systems, tickets, code modules) and their relationships, incrementally built by a background consolidation job that reads episodic memory and semantic memory and extracts

structured relations. Useful for multi-hop questions that pure vector similarity answers poorly (e.g., 'which services does this customer's failing payment depend on').

9.6 Memory Lifecycle Operations

Operation	What it does	Why it matters
Context compression	Summarizes older working-memory turns into a compact representation before they age out	Keeps prompts within budget without losing continuity
Memory consolidation	Periodically promotes durable facts from episodic to semantic memory	Prevents semantic memory from being polluted with one-off noise
Retrieval & ranking	Hybrid BM25 + vector similarity + recency scoring at query time	Pure vector similarity alone under-weights recency and exact-match facts
Conflict resolution	When new information contradicts stored semantic memory, the newer, higher-confidence source wins, with the old fact archived not deleted	Preserves auditability while keeping the active memory accurate
Expiration / forgetting	Tenant-configurable TTLs and a least-recently-used eviction policy for semantic memory beyond storage quota	Bounds cost and honors data-retention/regulatory requirements
Memory replay	Reconstructs an agent's exact context at any past timestamp from episodic memory	Essential for debugging and for compliance investigations
Indexing	Vector index (HNSW-class) plus inverted index for keyword search, kept in sync by the consolidation job	Hybrid retrieval needs both index types available

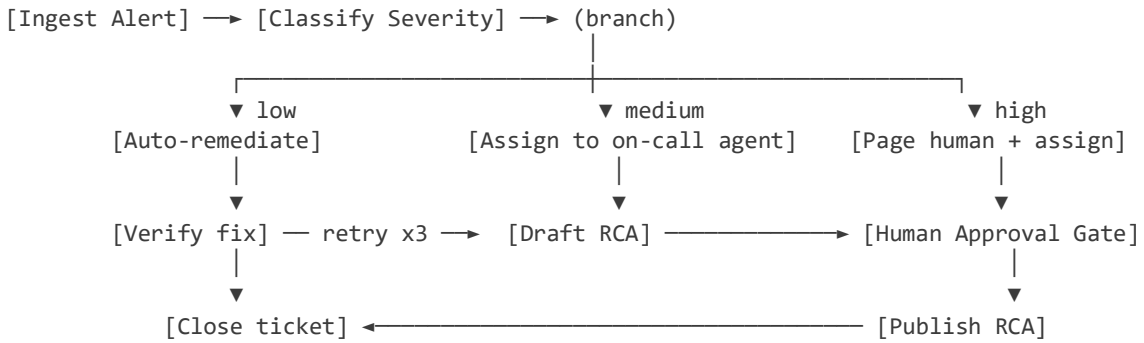
TRADE-OFF

Consolidating episodic into semantic memory automatically adds a background processing cost and a small risk of promoting incorrect information. The alternative — never auto-consolidating and requiring explicit writes to semantic memory — is safer but pushes real engineering burden back onto every agent author, which is exactly what AgentOS is trying to remove.

10. Workflow Engine

Multi-step, multi-agent processes are defined as either a DAG (for workflows whose structure is known ahead of time) or a state machine (for workflows with cycles, e.g., 'keep retrying with a different tool until success or budget exhausted'). Both compile to the same underlying durable-execution model as Temporal: every step's result is journaled before the next step runs, so a crash mid-workflow resumes rather than restarts.

Figure 10.1 — Example Workflow: Incident Triage



10.1 Core Constructs

- **Retries —** declarative backoff policy per step (fixed, exponential, jittered) with a max-attempts ceiling
- **Compensation —** saga-style rollback actions attached to steps with real-world side effects, so a failure halfway through can undo prior steps (e.g., cancel a partially-booked reservation)
- **Parallel execution —** fan-out to N branches (e.g., N sub-agents researching in parallel) with a join step that waits for all-or-quorum
- **Conditional branching —** expression-based routing on step outputs, including LLM-scored classifications
- **Loops —** bounded iteration constructs (e.g., 'retry with new strategy up to 5 times') with an explicit exit condition to avoid runaway agent loops
- **Timeouts —** per-step and per-workflow wall-clock limits that trigger a defined fallback path rather than hanging forever
- **Human approval —** a first-class step type that pauses the workflow, notifies a human via the Dashboard/Slack/email, and resumes on approval or rejection
- **Recovery & checkpointing —** inherited from the Workflow Engine's durable-execution journal described above

WHY THIS EXISTS

Long-running workflows are where every ad-hoc agent script eventually breaks: the process dies overnight, or a human takes two days to approve a step, and the whole in-memory state is gone. Building on a durable-execution model from day one — rather than retrofitting it — is why Temporal-style architectures were chosen as the inspiration here rather than a simple task queue.

ALTERNATIVE CONSIDERED

A simpler alternative is a pure DAG engine (Airflow-style) without state machines. This is easier to reason about but cannot express agent behaviors that need cycles, such as 'keep trying different tools until one works,' without ugly workarounds (self-triggering DAG runs).

11. Tool Execution Framework

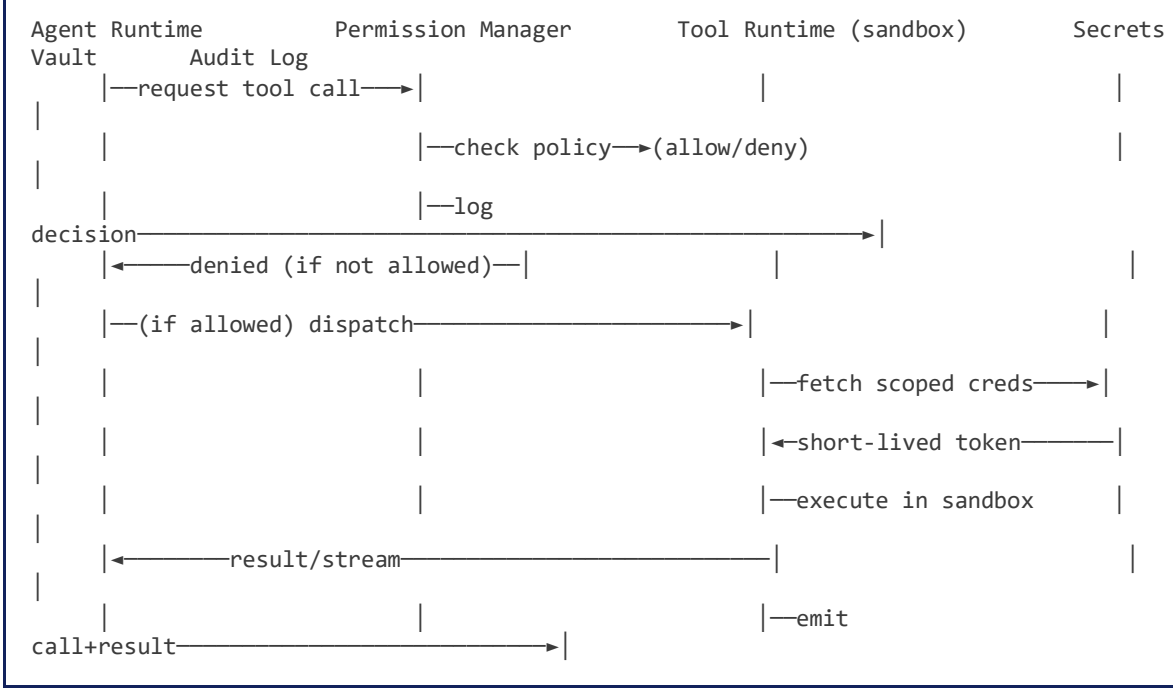
Tool calls are where an agent's decisions become real-world side effects, so the Tool Runtime is the most heavily isolated and audited component in AgentOS.

11.1 Supported Tool Categories

- Browser automation (headless browser pool)
- Database access (read/write, scoped by connection + row-level policy)
- Filesystem (scoped to a per-agent virtual filesystem)
- Cloud provider APIs (AWS/GCP/Azure, scoped by short-lived STS credentials)
- Email send/read
- GitHub (issues, PRs, code)
- Slack / chat platforms
- Docker / container management
- Terminal / shell execution

11.2 Execution Model

Figure 11.1 — Tool Call Path



11.3 Permission Model

Every agent identity has a declared allow-list of tool types and scopes in its manifest (e.g., 'github: read-only on repo X'; 'database: write on table Y for rows matching tenant_id'). The Permission Manager

evaluates this on every call — never trusting the agent's own claim about what it's allowed to do — and denies by default.

11.4 Sandbox & Isolation

Each tool call executes inside a micro-VM or gVisor-class sandbox with no network access beyond an explicit allow-list, no access to other agents' credentials, and resource limits (CPU, memory, wall-clock) enforced independently of the calling agent's own limits, so a runaway tool call cannot starve the agent process itself.

11.5 Secrets Management

Credentials are never given to the agent process. The Tool Runtime requests a short-lived, narrowly-scoped credential from a Secrets Vault (Vault/KMS-class) at call time, uses it, and lets it expire — the agent never sees the underlying long-lived secret.

11.6 Auditing & Logging

Every tool call's full input, output, the permission decision, and the identity of the requesting agent and workflow are written to the immutable audit log described in Section 19, enabling after-the-fact investigation of any real-world action an agent took.

> **WHY THIS EXISTS**
>
> Most publicized 'agent gone wrong' incidents are tool-execution incidents, not reasoning incidents — a bad SQL write, a wrong email sent, a destructive shell command. Isolating and auditing this layer is the single highest-leverage security investment in the whole platform.

12. LLM Abstraction Layer

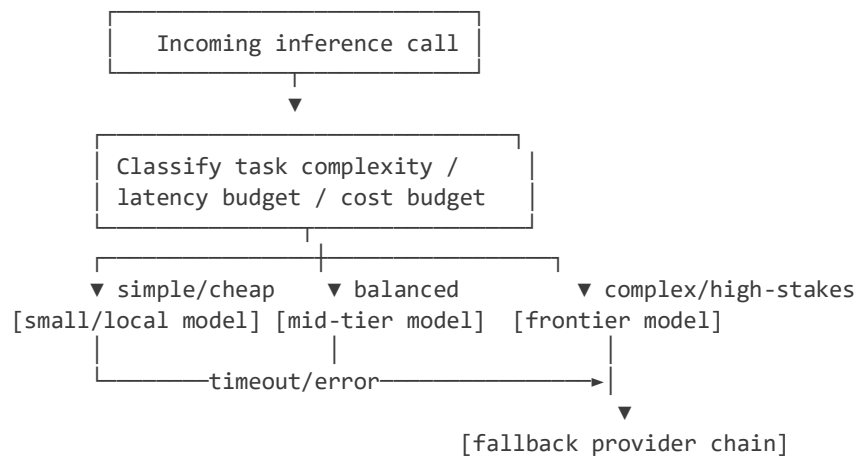
The LLM Gateway is the only component in AgentOS allowed to talk to a model provider. It presents one API to the rest of the system and hides provider differences (context limits, function-calling formats, streaming protocols) behind a normalized interface.

12.1 Supported Providers

- OpenAI
- Anthropic
- Google Gemini
- DeepSeek
- Mistral
- Meta Llama (self-hosted or hosted)
- Alibaba Qwen
- Locally-hosted open-weight models (vLLM/TGI-served)

12.2 Model Router

Figure 12.1 — Model Routing Decision



The Model Router selects a model per call using a policy that weighs task complexity classification, declared latency budget, and remaining cost budget (Section 21). On error, rate-limit, or timeout from the primary provider, it automatically retries against a configured fallback chain (e.g., primary frontier model → secondary frontier model → self-hosted open-weight model) so a single provider outage degrades rather than halts every agent.

12.3 Cost & Latency Optimization

- **Prompt caching —** reuse of cached prefill for repeated system prompts/context across calls where the provider supports it

- **Response caching —** semantic cache of prior (prompt, response) pairs for idempotent, repeatable queries
- **Load balancing —** spreading calls across multiple API keys/regions for a provider to avoid rate-limit cliffs
- **Automatic model selection —** downgrading to a cheaper model for low-stakes sub-tasks (e.g., classification) while reserving frontier models for final synthesis
- **Batching —** grouping non-latency-sensitive calls (e.g., nightly consolidation jobs) into provider batch APIs for lower cost

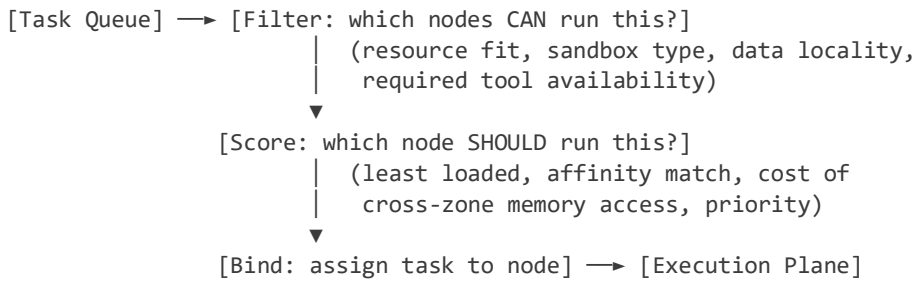
TRADE-OFF

Automatic model downgrading saves significant cost but risks quality regressions on tasks the complexity classifier misjudges. AgentOS mitigates this by letting agent manifests pin a minimum model tier for specific step types (e.g., 'never downgrade the final customer-facing response').

13. Scheduler

The Scheduler is modeled directly on the Kubernetes scheduler's two-phase design — filter then score — adapted with agent-specific resource dimensions: token budget, model-provider quota, and tool-concurrency locks in addition to CPU/memory.

Figure 13.1 — Scheduling Pipeline



13.1 Key Mechanisms

- **Priority queues —** per-tenant and per-workflow priority so a human-facing support agent preempts a background research agent
- **Preemption —** lower-priority agent instances can be checkpointed and paused (not killed) to free capacity for higher-priority work
- **Fair scheduling —** weighted fair-share across tenants prevents one noisy team from starving others of scheduler throughput
- **Resource allocation —** declared requests/limits per agent manifest for CPU, memory, token-budget-per-minute, and concurrent tool calls
- **Auto-scaling —** the agent pool for a given manifest scales horizontally based on queue depth and P95 latency
- **Affinity / anti-affinity —** e.g., co-locate an agent with its memory shard; keep two agents that write the same external resource off the same node to avoid lock contention
- **Recovery —** if a node fails health checks, its in-flight tasks are rescheduled from their last checkpoint onto healthy nodes

WHY THIS EXISTS

Reusing the Kubernetes scheduling model rather than inventing a new one means operators already understand the mental model (filter/score/bind, requests/limits, affinity rules), which lowers the adoption barrier for platform teams already running Kubernetes.

14. Distributed Architecture

AgentOS is designed to run across many nodes and, for large deployments, many geographic regions.

14.1 Cluster Management & Leader Election

Control-plane services (Scheduler, Workflow Engine coordinators) run as replicated instances with leader election via a Raft-based consensus store (etcd-class), so a leader failure triggers automatic failover without operator intervention.

14.2 Consensus & Distributed Locks

Agent state transitions that must not be double-applied (e.g., 'only one instance should pick up this task') use distributed locks backed by the same consensus store, following the same lease/fencing-token pattern used in Kubernetes to prevent split-brain writes.

14.3 Service Discovery & Replication

Every service registers itself with a discovery mechanism (built on the consensus store or a service mesh); data stores are replicated with configurable replication factor per tenant's durability requirements.

14.4 Sharding & Horizontal Scaling

Layer	Sharding key	Scaling approach
Task Queue	Tenant / priority class	Add partitions; consumer groups scale horizontally
Memory Plane	Agent/tenant ID	Range or hash sharding across storage nodes
Vector Store	Namespace (tenant/team)	Sharded ANN index with per-shard replicas
Event Store	Time + partition key	Kafka-style partitioning, log compaction for state topics

14.5 Failover & Geo-Distribution

Each region runs a full stack; cross-region replication of the Agent Registry and Event Store (asynchronous, eventually consistent) allows an agent to be resumed in a secondary region if its home region fails, at the cost of at-most a few seconds of lost progress since the last cross-region sync — an explicit, declared trade-off between durability and cross-region latency.

> **TRADE-OFF**
>
> Synchronous cross-region replication would eliminate that lost-progress window but would add tens to hundreds of milliseconds to every checkpoint write, unacceptable for latency-sensitive interactive

agents. AgentOS defaults to async replication with synchronous replication available as an opt-in, higher-cost tier for compliance-critical workloads.

15. Event-Driven Design

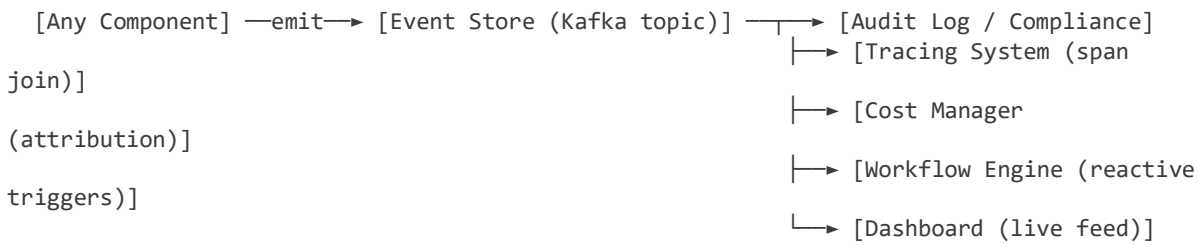
Every state change in AgentOS is emitted as an immutable event to the Event Store, which is both the audit trail and the coordination backbone for reactive workflows and dashboards.

15.1 Core Event Catalog

Event	Emitted when	Typical consumers
AgentRegistered	A new agent manifest is registered	Registry indexer, Dashboard
AgentStarted	An instance begins initialization	Scheduler, Tracing
TaskCreated	A task is enqueued for an agent	Scheduler, Cost Manager
ToolExecuted	A tool call completes (success or failure)	Audit Log, Permission analytics
MemorySaved	A write to any memory tier commits	Consolidation job, Tracing
CheckpointCreated	The Checkpoint Manager persists agent state	Recovery subsystem, Migration
TaskCompleted	A task finishes successfully	Workflow Engine, Dashboard, Cost Manager
TaskFailed	A task exhausts retries or hits a fatal error	Workflow Engine (compensation), Alerting
AgentScaled	The pool for a manifest scales in/out	Dashboard, Cost Manager
AgentTerminated	An instance shuts down (graceful or crash)	Garbage collector, Alerting

15.2 Event Flow

Figure 15.1 — Event Flow Through the Platform



WHY THIS EXISTS

An event-sourced core means the Dashboard, the Audit Log, the Cost Manager, and the Workflow Engine's reactive triggers are all just independent consumers of one durable stream, rather than each needing bespoke write paths into every producing service.

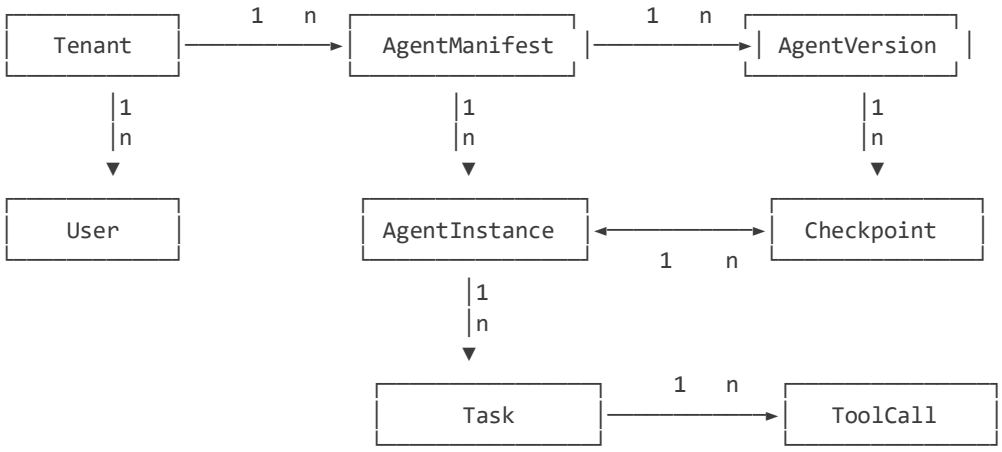
16. Database Design

AgentOS deliberately uses more than one storage engine, each matched to its access pattern, rather than forcing everything into one general-purpose database.

Data	Store type	Rationale
Agent Registry / manifests / versions	Relational (Postgres-class)	Strong schema, transactional updates, mature tooling
Event Store	Log-structured (Kafka + cold-tier object storage)	Append-only, high throughput, replayable
Episodic memory	Columnar/document store	Efficient time-range scans, flexible schema per agent type
Semantic memory	Vector store (HNSW-class ANN index)	Similarity search at scale
Knowledge graph	Graph database (Neo4j-class)	Multi-hop relationship queries
Working memory	In-memory KV (Redis-class)	Sub-millisecond reads/writes, TTL support
Checkpoints / large artifacts	Object storage (S3-class)	Cheap, durable, versioned blob storage

16.1 Core Relational Schema (Simplified ER Diagram)

Figure 16.1 — ER Diagram: Registry & Execution Core



16.2 Indexing, Partitioning, Caching

- **Indexes —** B-tree indexes on tenant_id + created_at for every high-cardinality table; HNSW indexes per-namespace for vector search
- **Partitioning —** event and episodic-memory tables partitioned by time (daily/weekly) so retention expiry is a cheap partition drop, not a row-by-row delete

- **Caching —** read-through cache in front of the Agent Registry for hot manifest lookups; working memory itself is the cache tier for context
- **Replication —** synchronous replicas for the relational store (registry correctness matters); asynchronous replicas for episodic/event stores (throughput matters more than zero-lag reads)
- **Backup & retention —** point-in-time recovery for the relational store; tenant-configurable retention windows for episodic memory and events, enforced by scheduled partition expiry

17. APIs

17.1 REST API — Representative Endpoints

Method & Path	Purpose
POST /v1/agents	Register a new agent manifest
GET /v1/agents/{id}	Fetch an agent manifest and its versions
POST /v1/agents/{id}/tasks	Submit a task to a running/queued agent
GET /v1/tasks/{id}	Poll task status/result
POST /v1/workflows	Register a workflow definition
POST /v1/workflows/{id}/runs	Start a workflow run
POST /v1/workflows/runs/{id}/approve	Resolve a human-approval gate
GET /v1/memory/{agentId}/semantic	Query semantic memory
GET /v1/traces/{taskId}	Fetch the full execution trace for a task
GET /v1/cost/summary	Query cost attribution by tenant/agent/workflow

Example: Submit a Task

Request / Response Example

```
POST /v1/agents/support-triage/tasks
{
  "input": "Customer reports failed payment on order 88213",
  "priority": "high",
  "budget": { "max_tokens": 50000, "max_usd": 0.75 },
  "context_refs": ["ticket:88213"]
}

202 Accepted
{
  "taskId": "task_9f21ac",
  "status": "queued",
  "streamUrl": "wss://agentos/v1/tasks/task_9f21ac/stream"
}
```

17.2 gRPC API

High-throughput internal calls (Scheduler↔Runtime, Runtime↔LLM Gateway, Runtime↔Memory Engine) use gRPC with protobuf-defined service contracts for lower serialization overhead and native streaming support, reserving REST for external, human/SDK-facing interactions where JSON's debuggability matters more than raw throughput.

17.3 WebSocket API

Task submission responses include a WebSocket stream URL so clients can receive incremental agent output (tokens, intermediate tool-call events, status transitions) rather than polling — mirroring how streaming chat completions work today, extended to include lifecycle and tool events, not just text tokens.

TRADE-OFF

Supporting three protocols (REST, gRPC, WebSocket) is more surface area to maintain than one, but each serves a genuinely different consumer: REST for external integration simplicity, gRPC for internal throughput, WebSocket for real-time UX. Collapsing to just REST-with-polling would visibly degrade the interactive dashboard experience.

18. Security

18.1 Authentication & Authorization

- **Authentication —** humans via OAuth2/OIDC through the Identity Provider; services and agents via mutual TLS or short-lived signed JWTs issued at instance start
- **RBAC —** coarse-grained roles (viewer, operator, admin) scoped per tenant, controlling access to platform operations (deploy an agent, view traces, approve workflows)
- **ABAC —** fine-grained, attribute-based rules for tool and data access (e.g., 'this agent may write to database rows where tenant_id matches its own tenant'), evaluated by the Permission Manager on every tool/memory call
- **API Keys —** scoped, rotatable keys for programmatic SDK access, with per-key rate limits and audit trails

18.2 Encryption & Secrets

- TLS in transit for all internal and external traffic
- Encryption at rest for the relational store, object storage, and vector store
- Secrets never given directly to agent processes — issued just-in-time as short-lived, scoped credentials by the Secrets Vault (Section 11.5)

18.3 Audit Logging

Every authentication decision, permission check, tool call, and workflow approval is written to an append-only audit log, independently retained from the operational Event Store so audit history cannot be altered by application-level bugs or malicious agent behavior.

18.4 Zero Trust & Sandboxing

No component trusts another purely on network location: every call is authenticated and authorized, including calls between the Agent Runtime and the Memory Engine. Tool execution runs in micro-VM/gVisor-class sandboxes with default-deny network policy, as detailed in Section 11.4.

18.5 Prompt Injection Protection

Because tool outputs and retrieved memory can contain adversarial instructions embedded in text (e.g., a webpage telling the agent to exfiltrate secrets), AgentOS treats all tool/retrieval output as untrusted data, never as instructions: the LLM Gateway tags such content distinctly in the prompt structure, the Permission Manager still enforces tool scopes regardless of what the model 'decides' to do, and high-risk tool calls (sending money, deleting data, external communication) can be configured to require a human-approval workflow gate even if the model requests them autonomously.

> **WHY THIS EXISTS**
>
> Prompt injection is fundamentally a permission-boundary problem, not a prompting problem — no amount of prompt engineering reliably stops a model from being manipulated by adversarial input.

AgentOS's answer is to make sure that even a fully manipulated model cannot exceed the tool permissions granted to its identity, and that unusually consequential actions still require a human checkpoint.

19. Observability

19.1 Logging, Metrics, Tracing

- **Structured logging —** every component emits structured logs correlated by task ID and trace ID
- **Metrics —** Prometheus-style metrics for queue depth, scheduling latency, token throughput, tool-call error rate, and per-tenant cost burn rate
- **Distributed tracing —** every task gets one trace spanning Gateway → Scheduler → Runtime → LLM Gateway → Memory Engine → Tool Runtime, so a slow or failed task can be diagnosed span by span

19.2 Dashboards & Health

The Dashboard surfaces live agent execution, per-workflow success/failure rates, cost attribution by team/agent, and cluster health (node capacity, scheduler queue depth). Health checks (liveness/readiness) run on every stateless service and on the Agent Runtime's ability to reach the LLM Gateway and Memory Engine.

19.3 Alerting

Threshold- and anomaly-based alerts on error rate, cost burn rate, latency, and tool-permission denials, routed to the same on-call tooling (PagerDuty-class) that platform teams already use for other services.

19.4 Agent & Execution Replay

Because every memory read/write and tool call is event-sourced, AgentOS can reconstruct the exact sequence of prompts, retrievals, and tool calls that produced any past result — the same 'replay' capability that made the Event Store worth the operational cost in Section 15, now paying off directly for debugging and incident postmortems.

20. Failure Handling

Failure mode	Detection	Response
Node failure	Missed heartbeats / failed health checks	Scheduler reschedules in-flight tasks from last checkpoint onto healthy nodes
Agent crash	Runtime process exit / exception	Automatic restart from last checkpoint, up to a configured retry ceiling, then escalate to dead-letter
LLM timeout	Gateway call exceeds latency budget	Automatic fallback to secondary provider/model per Section 12.2's routing chain
Tool failure	Non-zero exit / error response from Tool Runtime	Retry per the workflow step's backoff policy; compensation action if retries exhausted
Database outage	Connection/health check failure	Circuit breaker opens; reads served from cache where possible; writes queued and replayed on recovery
Memory corruption	Checksum/schema validation failure on read	Fallback to last known-good checkpoint; corrupted record flagged for investigation, not silently overwritten
Network partition	Consensus store quorum loss	Minority partition stops accepting writes (favors consistency over availability for control-plane state)

20.1 Retry Strategy & Circuit Breakers

Retries use exponential backoff with jitter, bounded by a per-step max-attempts limit declared in the workflow definition. Circuit breakers wrap every external dependency (LLM providers, tools, databases) so a failing dependency degrades gracefully — opening the circuit and failing fast — rather than cascading latency into every caller.

20.2 Dead-Letter Queues & Compensation

Tasks that exhaust retries move to a dead-letter queue for human triage rather than being silently dropped. Workflows with real-world side effects attach compensation steps (Section 10.1) so a failure partway through a multi-step process actively undoes what it can, rather than leaving the world in an inconsistent half-completed state.

> **TRADE-OFF**
> Favoring consistency over availability during a network partition (control-plane state stops accepting writes in the minority partition) means brief unavailability during a rare partition event, in exchange for never double-scheduling the same agent task on two sides of a split cluster — an acceptable trade for a system whose tool calls can have real, non-idempotent side effects.

21. Cost Optimization

Token and compute spend is treated as a scheduling and governance dimension, not an afterthought reported at the end of the month.

21.1 Mechanisms

- **Model selection —** the Model Router (Section 12.2) defaults to the cheapest model that meets a task's declared quality bar
- **Prompt & response caching —** avoids repeat spend on identical or near-identical calls
- **Token optimization —** the Prompt Manager tracks token cost per template version, flagging regressions when a prompt change inflates average token usage
- **Batch execution —** non-interactive workloads (consolidation, nightly analytics) are queued into provider batch APIs at lower per-token cost
- **Memory compression —** working-memory compression (Section 9.6) reduces the context sent per call, directly reducing token spend
- **Scheduling optimization —** the Scheduler considers a task's remaining cost budget as a scheduling input, throttling lower-priority work before it blows through a tenant's budget
- **Idle detection —** the sleeping-agent lifecycle state (Section 7.4) ensures agents waiting on external events consume zero compute

21.2 Cost Governance

Every agent manifest declares a cost budget (max tokens, max USD) per task and per time window; the Cost Manager enforces these budgets in real time, and the Dashboard attributes spend down to team, agent, workflow, and even individual prompt-template version, so cost regressions are traceable to a specific change rather than showing up only as a bigger invoice.

22. Plugin & Marketplace

AgentOS extends through a plugin ecosystem modeled on VS Code extensions and container image registries combined: versioned, sandboxed packages that add capability without modifying core.

22.1 Package Types

Package type	Contains	Example
Agent package	A full agent manifest + prompts + default tool bindings	'Code Review Agent v2'
Tool package	A tool implementation + its permission schema	'Snowflake connector'
Memory module	A pluggable backend for a memory tier	'Pinecone vector store adapter'
Workflow template	A reusable, parameterizable workflow definition	'Incident triage template'
Auth plugin	An identity-provider integration	'Okta SSO connector'
Deployment plugin	A target-environment adapter	'On-prem air-gapped deploy profile'

22.2 Versioning, Publishing, Trust

- **Versioning —** semantic versioning with declared compatibility ranges against core AgentOS API versions
- **Publishing —** a review pipeline that statically scans for over-broad permission requests and known-vulnerable dependencies before a package is listed
- **Trust model —** tiered trust levels — Anthropic/first-party verified, community-reviewed, and unreviewed — surfaced clearly in the marketplace UI, with unreviewed packages defaulting to the most restrictive sandbox profile

WHY THIS EXISTS

A marketplace only earns trust if the platform itself enforces a floor on what an unreviewed plugin can do — sandboxing and default-deny permissions apply to marketplace tools exactly as they do to first-party ones (Section 11), so a malicious or buggy plugin can't become a security incident.

23. Future Roadmap

- **Agent swarms —** large, dynamically-formed groups of homogeneous agents coordinating on a single goal via the shared-memory and pub/sub primitives already in the platform
- **Self-healing systems —** the platform detecting a recurring class of tool or workflow failure and automatically proposing (or applying, with approval) a workflow-definition patch
- **Federated clusters —** multiple independently-operated AgentOS clusters (e.g., across business units or partner companies) interoperating through a federation API without sharing a control plane
- **Multi-cloud —** first-class scheduling across cloud providers, not just regions, for cost arbitrage and resilience
- **On-device execution —** a lightweight Agent Runtime profile for edge/on-device deployment where data cannot leave a local boundary
- **Federated memory —** cross-cluster semantic memory sharing with privacy-preserving aggregation
- **Self-improving agents —** structured feedback loops where an agent's own outcome history informs prompt or tool-selection tuning, gated by evaluation and approval workflows
- **Agent evolution / learning agents —** longer-horizon research direction: agents whose policies adapt over time based on accumulated episodic memory, within the same auditability and permission guarantees as the rest of the platform

> **NOTE**
> These are explicitly framed as directions, not commitments in the core v1 design — each depends on the foundational lifecycle, memory, and permission systems in Sections 6-11 being solid first.

24. Comparison

System	Strength	Weakness relative to AgentOS	Relationship to AgentOS
LangChain / LangGraph	Rich composition primitives, huge integration ecosystem	No cluster scheduler, multi-tenant memory, or platform-level permission model	Can run as an agent-definition SDK targeting the AgentOS runtime
CrewAI	Simple mental model for role-based agent teams	Not designed for durable, crash-recoverable, long-running execution	Crew patterns map onto AgentOS's shared-memory + workflow primitives
AutoGen	Strong multi-agent conversation patterns	Conversation-centric, weak on state durability and tool sandboxing	Conversation patterns implementable as an AgentOS communication pattern
Semantic Kernel	Good enterprise .NET/C# integration	SDK-shaped, not a multi-tenant runtime	Could act as a client SDK against the AgentOS API
Temporal	Best-in-class durable workflow execution	No concept of 'agent' as a resource, no LLM gateway, no agent memory	AgentOS's Workflow Engine borrows its durable-execution model directly
Ray	Excellent distributed compute scheduling for ML workloads	General-purpose compute scheduler, not agent-lifecycle- or tool-permission-aware	Could serve as a low-level execution substrate under the Agent Runtime
Apache Airflow	Mature batch DAG scheduling, huge operator ecosystem	Batch-oriented, not built for long-lived, interactive, stateful agents	Inspiration for the DAG mode of the Workflow Engine
Kubernetes	The gold-standard resource scheduler and lifecycle manager	No concept of memory, tools, LLMs, or agent-specific permissions	AgentOS's Scheduler and lifecycle model are directly modeled on it, one layer up the stack
Docker	Standardized packaging and isolation for a unit of compute	No orchestration, no concept of an agent at all	Analogous inspiration for the Agent Manifest as a packaging format

The honest positioning: AgentOS is not a replacement for Kubernetes, Temporal, or Kafka — it is built on top of, or alongside, exactly those systems, adding the agent-specific layer (memory, tool permissions, LLM routing, agent lifecycle) that none of them provide natively. Its competitive set is the growing list of agent frameworks, which it aims to absorb as SDKs that target its runtime rather than compete with as alternatives.

25. Real-World Use Cases

25.1 Autonomous Software Engineering

Execution Flow

```
[Ticket ingested] → [Planner agent decomposes task]
        ├→ [Coding agent: branch + implement] → [Test-runner tool] → pass?
        │                                                              │no
        │                                                              ▼
        │                                                  [Debug loop, bounded
retries]
        └→ [Review agent: static analysis + diff review] → [Human Approval Gate] →
[Merge tool]
```

Coding and reviewing agents run as separate manifests with distinct tool permissions (write access to a branch, no direct merge rights); the merge itself always sits behind a human-approval workflow gate, and every diff and test run is fully traced for later audit.

25.2 AI Research Teams

Execution Flow

```
[Research question] → [Lead agent splits into sub-questions]
        ├→ [Sub-agent A: literature search] ┐
        ├→ [Sub-agent B: data analysis]      ├→ [Shared session memory] → [Synthesis
agent] → [Report]
        └→ [Sub-agent C: experiment design] ┘
```

Parallel fan-out workflow (Section 10.1) with sub-agents writing to shared/global memory (Section 9.4); the synthesis step reads the consolidated shared context rather than replaying every sub-agent's full transcript.

25.3 Customer Support Automation

Execution Flow

```
[Ticket created] → [Triage agent classifies] → (branch)
     low complexity → [Resolution agent] → [Auto-reply tool] → [Close]
     high complexity → [Escalation] → [Sleeping: await human reply] → [Wake on reply]
→ [Resume]
```

Demonstrates the sleeping/wake-up lifecycle state (Section 7.4) directly — an agent waiting days for a customer response consumes no compute, and semantic memory retains prior interaction history for continuity when it wakes.

25.4 Cybersecurity Operations

Execution Flow

```
[SIEM alert event] → [Triage agent] → severity?
   low  → [Auto-remediate tool] → [Verify] → [Close]
   high → [Page human] → [Forensics agent: read-only tool access] → [Human Approval] →
[Contain/remediate]
```

High blast-radius tools (isolate host, revoke credentials) require the human-approval gate even under time pressure; every tool call is logged to the immutable audit trail for post-incident and regulatory review.

25.5 Healthcare Diagnostics Support

Execution Flow

```
[Patient data ingested] → [Analysis agent: read-only EHR tool access]
        → [Draft differential] → [Human clinician Approval Gate] → [Attach to chart]
```

The agent never has write access to the medical record — only the human-approved output is committed — and ABAC policies restrict which patient records the agent identity may read, aligned with regulatory (e.g., HIPAA-style) access requirements.

25.6 Enterprise Knowledge Management

Execution Flow

```
[Employee query] → [Retrieval agent: semantic + knowledge graph search]
        → [Synthesis agent] → [Answer + citations] → [Feedback loop updates semantic
memory ranking]
```

Leans on the full Memory Plane (Section 9): semantic memory for document embeddings, the knowledge graph for cross-document entity relationships, and consolidation to keep the corpus current as new documents arrive.

25.7 Financial Analysis

Execution Flow

```
[Analyst request] → [Data-gathering agent: scoped read-only DB + market-data tool]
        → [Modeling agent] → [Compliance check step] → [Human Approval] → [Publish
report]
```

Cost budgets (Section 21) matter acutely here given expensive market-data API calls; the Model Router downgrades routine data-cleaning steps to a cheaper model while reserving the frontier model for final analysis synthesis.

25.8 Scientific Research

Execution Flow

```
[Hypothesis] → [Literature agent] → [Experiment-design agent] → [Lab-automation tool
(sandboxed)]
        → [Results analysis agent] → [Update knowledge graph] → [Propose next
hypothesis]
```

A long-horizon workflow that can span weeks; checkpointing and the sleeping lifecycle state let the workflow pause between physical experiment cycles without holding compute, resuming when lab-automation tool results arrive.